

01 · Datos y rendimientos

Quant-Backtester · material de estudio

Todo backtest empieza con una tabla de precios y acaba en una tabla de rendimientos. En este capítulo construyes el paso intermedio y entiendes por qué cada decisión importa: qué precio usar, cómo medir el cambio de un día al siguiente y qué hacer cuando los datos llegan sucios. Un fallo aquí no rompe nada de forma visible: produce un backtest que se ejecuta sin errores y da cifras falsas.

El capítulo cubre tres TODO de `src/data.py`: **1.1 simple_returns**, **1.2 log_returns** y **1.3 clean_prices**. El motor del capítulo 02 llama a `simple_returns` en cada backtest, así que sin el TODO 1.1 no pasa ningún test del motor completo. El checkpoint es `tests/test_data.py` entero.

1. Intuición

Rendimientos, no precios. Que una acción valga 400 USD y otra 40 USD no dice nada comparable. Lo que te importa es cuánto cambia tu dinero si tienes el activo en cartera, y eso es un cociente entre el precio de hoy y el de la sesión anterior. Los rendimientos no tienen unidades, se comparan entre activos y son lo que se multiplica por una posición para obtener un P&L. Salvo las señales, casi todo lo que harás después trabaja sobre rendimientos.

Precios ajustados. El precio que cotiza no mide lo que gana quien tiene la acción. Dos sucesos rompen la relación:

- Un *split* 4:1 convierte cada acción de 500 USD en cuatro de 125 USD. Tu riqueza no cambia, pero la serie de cotizaciones cae un 75 % de un día para otro.
- Un dividendo de 2 USD en una acción de 50 USD se paga en efectivo y, el día ex-dividendo, el precio cae aproximadamente esos 2 USD. La cotización marca -4 %; tú no has perdido nada, lo tienes en caja.

Un backtest con precios sin ajustar vería una caída del 75 % que nunca ocurrió y penalizaría sistemáticamente a las empresas que reparten dividendos. El precio ajustado corrige toda la historia *anterior* a cada suceso para que el cociente entre dos sesiones consecutivas sea el rendimiento total: precio más dividendos reinvertidos. `download_prices` llama a `yfinance` con `auto_adjust=True`, así que la columna `Close` que se guarda en `data/raw/prices.parquet` ya viene ajustada por splits y dividendos (por eso no hay columna `Adj Close`), y `prices_meta.json` lo deja anotado.

Dos consecuencias prácticas. Primera: los niveles de una serie ajustada no son precios que nadie pague. Yahoo recalcula hacia atrás toda la historia cada vez que hay un dividendo nuevo, de modo que dos descargas separadas unos meses dan niveles distintos para la misma fecha y, salvo redondeos, los mismos rendimientos. Por eso el proyecto guarda una foto en parquet con la fecha de descarga. Segunda: sobre una serie ajustada trabaja con cocientes y rendimientos, nunca con niveles absolutos. "Comprar si el precio supera 100 USD" no significa nada si el 100 USD de 2012 ha sido reescalado.

Dos formas de medir el mismo cambio. Si un precio pasa de 100 a 110, el rendimiento simple es $110/100 - 1 = 0,10$ y el logarítmico es $\ln(110/100) \approx 0,0953$. Para los movimientos diarios típicos casi coinciden. La diferencia que importa está en cómo se

agregan:

- **Entre activos, el mismo día** (una cartera): se usan los simples. El rendimiento de una cartera es exactamente la media ponderada de los rendimientos simples de sus activos. Con logarítmicos esa igualdad es falsa.
- **En el tiempo, el mismo activo**: los logarítmicos se suman. Los simples no se suman: se componen, multiplicando los factores $1 + r$.

El motor del capítulo 02 usa rendimientos simples porque cada día agrega *entre activos* (el P&L de la cartera es una suma de posiciones por rendimientos). La agregación *en el tiempo* la resuelve con un producto acumulado (TODO 2.6), no con sumas.

Los datos llegan sucios. Huecos (NaN) porque un activo no cotizó o la fuente falló; fechas repetidas al concatenar descargas o por correcciones del proveedor; precios iguales o menores que 0, que en acciones y ETF son imposibles y por tanto son errores de carga; activos que empiezan a cotizar en fechas distintas. `clean_prices` trata cada caso con una regla explícita, y todas obedecen al mismo principio: **para rellenar un dato de la fecha t solo puedes usar información disponible en t** . Por eso se rellena hacia delante con el último precio conocido, con un límite, y nunca hacia atrás.

Sesgo de supervivencia. El universo de `config.TICKERS` se ha elegido hoy. AAPL, MSFT, JPM y XOM son empresas de las que en 2010 nadie sabía que seguirían siendo enormes quince años después. Las que quebraron, fueron absorbidas o salieron de los índices no están en la lista, y `yfinance` ni siquiera sirve precios de muchos valores deslistados. Elegir los activos sabiendo cuáles sobrevivieron es usar información del futuro en el diseño del experimento. Ninguna función de limpieza lo corrige: haría falta un universo *point-in-time* (los componentes que tenía el índice en cada fecha, incluidos los que después desaparecieron). Aquí se acepta como limitación documentada en el README; tenla presente cada vez que leas una cifra de rentabilidad de este proyecto.

2. Formalización

Notación: $P_{\{i,t\}}$ es el cierre ajustado del activo i en la sesión t . El índice t cuenta sesiones (filas de la tabla), no días naturales: "ayer" es la sesión anterior aunque entre medias haya un fin de semana o un festivo. Cuando solo hay un activo se omite i .

El rendimiento simple es

$$r_t = \frac{P_t}{P_{t-1}} - 1$$

y el logarítmico

$$\ell_t = \ln\left(\frac{P_t}{P_{t-1}}\right) = \ln(1 + r_t)$$

Ambos necesitan dos precios. Si falta P_t o P_{t-1} , el rendimiento no existe y vale NaN; en la primera fila no existe nunca. No se rellena nada: decidir qué hacer con los huecos es trabajo de `clean_prices`, no de la fórmula.

Agregación en el tiempo. Al encadenar T sesiones, los precios intermedios se cancelan:

$$\frac{P_T}{P_0} = \prod_{t=1}^T (1 + r_t)$$

$$\ln\left(\frac{P_T}{P_0}\right) = \sum_{t=1}^T \ell_t$$

Compruébalo a mano:

t	P _t	r _t	ℓ _t	Π (1 + r)
0	100,0	NaN	NaN	1,000
1	110,0	0,1000	0,0953	1,100
2	99,0	-0,1000	-0,1054	0,990
3	99,0	0,0000	0,0000	0,990
4	108,9	0,1000	0,0953	1,089

La suma de los ℓ_t es 0,08526 = ln 1,089 (con los valores redondeados de la tabla sale 0,0852). La suma de los r_t es 0,10, pero el activo ha ganado un 8,9 %: sumar rendimientos simples a lo largo del tiempo es un error. Con simples, lo correcto es el producto de la última columna.

Agregación entre activos. Si repartes un capital V con pesos w_i que suman 1, cada parte crece por su factor 1 + r_{i,t} y el total pasa a valer V · (1 + Σ w_i · r_{i,t}). Por tanto

$$r_t^{\text{cartera}} = \sum_{i=1}^N w_i \cdot r_{i,t}$$

mientras que con logaritmos

$$\ell_t^{\text{cartera}} = \ln\left(1 + \sum_{i=1}^N w_i \cdot r_{i,t}\right) \neq \sum_{i=1}^N w_i \cdot \ell_{i,t}$$

Ejemplo: cartera 50/50 en A y B, ambos a 100 al empezar.

Activo	Precio final	r	ℓ
A	110	0,1000	0,0953
B	90	-0,1000	-0,1054
Cartera 50/50	100	0,0000	0,0000
Media ponderada de A y B		0,0000	-0,0050

La media ponderada de los simples reproduce la cartera; la de los logarítmicos se equivoca en 50 puntos básicos en un solo día.

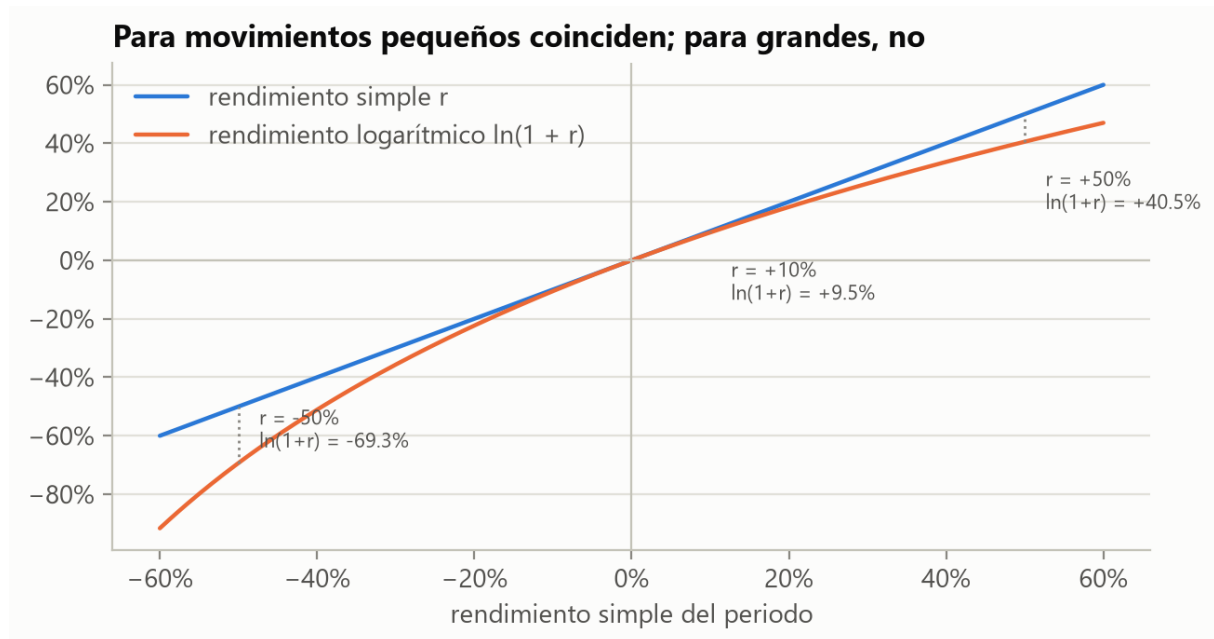
La aproximación ln(1 + r) ≈ r. El desarrollo de Taylor alrededor de 0 da

$$\ln(1 + r) \approx r - \frac{r^2}{2}$$

así que la diferencia r - ℓ es aproximadamente r²/2 y siempre positiva: el logarítmico queda por debajo del simple tanto en subidas como en bajadas.

r	ℓ = ln(1 + r)	r - ℓ (bps)
0,1 %	0,09995 %	0,005
1 %	0,995 %	0,5
5 %	4,879 %	12,1
10 %	9,531 %	46,9

r	$\ell = \ln(1 + r)$	$r - \ell$ (bps)
-10 %	-10,536 %	53,6
-50 %	-69,315 %	1931



Rendimiento logarítmico frente a rendimiento simple: casi iguales cerca de cero, se separan en los movimientos grandes, sobre todo en las caídas

Con rendimientos diarios de acciones y ETF, casi siempre por debajo del 2 % en valor absoluto, la diferencia no llega a 2 bps al día, y por eso en cálculos rápidos se usan indistintamente. Pero el error es sistemático y se acumula. Promediando la aproximación:

$$\bar{\ell} \approx \bar{r} - \frac{\sigma^2}{2}$$

donde σ es la volatilidad de los rendimientos simples. Es el *volatility drag*: lo que crece tu dinero (la media de los logarítmicos) es menor que la media aritmética de los simples, y la brecha aumenta con la volatilidad. Caso extremo: +50 % y después -50 %. La media de los simples es 0; el capital final es $1,5 \times 0,5 = 0,75$; la suma de los logarítmicos es $0,405 - 0,693 = -0,288 = \ln 0,75$.

Ajustes. Si un split k:1 tiene efecto en la sesión t, todos los precios anteriores se dividen entre k:

$$P_s^{\text{adj}} = \frac{P_s}{k}$$

y si un dividendo D tiene fecha ex-dividendo t, todos los precios anteriores se multiplican por un factor algo menor que 1:

$$P_s^{\text{adj}} = P_s \cdot \left(1 - \frac{D}{P_{t-1}}\right)$$

en ambos casos para toda $s < t$. Ejemplo con $P_{t-1} = 50$, $D = 2$ y $P_t = 48$ (el precio cae exactamente el dividendo): el factor es 0,96, el precio ajustado de t-1 pasa a 48 y el rendimiento ajustado es $48/48 - 1 = 0$, frente al -4 % de la cotización. Los factores se acumulan: un precio de 2010 lleva multiplicados los factores de todos los sucesos posteriores.

Reglas de limpieza. Para una tabla en bruto, con las filas en el orden en que llegaron:

- Fechas repetidas: se conserva la última aparición (la versión más reciente del dato) y después se ordena por fecha.
- Si $P_{i,t} \leq 0$, se trata como dato ausente: NaN.
- Si en la fecha t todos los $P_{i,t}$ son NaN, la fila se elimina: no fue un día de mercado para ninguno de tus activos.
- Relleno hacia delante con límite L (`max_ffill`, 5 por defecto en `config.MAX_FFILL_DAYS`): en cada columna, los L primeros NaN consecutivos de un hueco toman el último precio válido; del $(L+1)$ -ésimo en adelante, el hueco se queda en NaN.
- Fecha común de inicio: la primera sesión en la que todos los activos tienen precio, es decir, la más tardía de las primeras fechas válidas de cada columna.

$$t_0 = \max_i (\min \{t : P_{i,t} \neq \text{NaN}\})$$

Por qué un límite y no rellenar siempre: uno o dos días sin dato (una suspensión breve, un fallo de la fuente) se resuelven razonablemente con el último precio conocido, que es lo que usarías para valorar tu posición. Un hueco de semanas es otra cosa (exclusión de cotización, cambio de ticker, datos rotos): inventar un precio plano durante semanas falsea la volatilidad, las medias móviles y cualquier señal calculada en ese tramo. Dejar NaN hace visible el problema para que lo investigues.

Por qué nunca hacia atrás: con la serie [10, NaN, 12], el relleno hacia delante da [10, 10, 12] y el día 2 solo "sabe" lo que ya había pasado. El relleno hacia atrás da [10, 12, 12]: el día 2 ya muestra un precio que no existió hasta el día 3. Una señal calculada al cierre del día 2 anticiparía la subida. Es look-ahead, aunque sea "solo un día".

Ejemplo completo. Tabla en bruto, en el orden de llegada:

Llegada	Fecha	A	B
1	2024-01-03	10,0	20,0
2	2024-01-02	NaN	19,5
3	2024-01-04	10,2	20,4
4	2024-01-04	10,3	20,5
5	2024-01-05	NaN	NaN
6	2024-01-08	-1,0	20,7

Tras limpiar con el límite por defecto:

Fecha	A	B
2024-01-03	10,0	20,0
2024-01-04	10,3	20,5
2024-01-08	10,3	20,7

Comprueba cada paso: el 2024-01-04 se queda con la llegada 4, la última; el 2024-01-05 desaparece porque está entero en NaN; el -1,0 se convierte en NaN y se rellena con 10,3; y el 2024-01-02 se recorta porque A aún no cotizaba, aunque B sí. Fíjate en que A no se rellena hacia atrás con el 10,0 del día siguiente.

Y el límite, con una serie de juguete y $L = 2$: [5, NaN, NaN, NaN, 7] queda [5, 5, 5, NaN, 7]. El tercer NaN seguido ya no se rellena.

3. Del papel al código

Antes de los TODO, cuatro ideas de pandas que necesitas en los tres.

Alineación por etiquetas. La aritmética entre dos objetos de pandas empareja por etiquetas (fechas en el índice, nombres en las columnas), no por posición. Si las etiquetas no coinciden, el resultado tiene la unión de etiquetas, con NaN donde falta un lado:

```
a = pd.Series([1.0, 2.0, 3.0], index=["lun", "mar", "mié"])
b = pd.Series([10.0, 20.0, 30.0], index=["mar", "mié", "jue"])
a + b      # jue NaN, lun NaN, mar 12.0, mié 23.0
```

Es una red de seguridad: impide combinar por error el dato de un día con el de otro. Si conviertes a NumPy (`to_numpy()`, `.values`) pierdes las etiquetas y nadie comprueba que las filas casen; además, los tests comparan el índice del resultado. Mientras trabajas en pandas, quédate en pandas.

shift mueve valores, no fechas. `shift(n)` devuelve un objeto con el mismo índice y los valores desplazados `n` filas; las `n` primeras quedan NaN, o con el valor que pases en `fill_value`. Con `n` negativo desplaza hacia atrás y trae el futuro al presente:

```
s = pd.Series([10, 11, 12], index=["lun", "mar", "mié"])
s.shift(1)      # lun NaN, mar 10.0, mié 11.0
s.shift(-1)     # lun 11.0, mar 12.0, mié NaN
```

Tras `shift(1)`, la fila de cada fecha contiene el valor de la fila anterior, es decir, de la sesión anterior. No lo confundas con `shift(freq=...)`, que mueve las etiquetas del índice en el calendario y rompería la alineación en cuanto hubiera un festivo.

Los NaN se propagan. Cualquier operación aritmética con un NaN da NaN: $5 + \text{NaN}$, $0 \times \text{NaN}$, $\text{NaN} / 3$. Las reducciones (`sum`, `mean`) los ignoran por defecto. Para los rendimientos, la propagación es justo lo que quieres: si falta un precio, el rendimiento sale NaN sin que escribas nada especial.

No modifiques la entrada. Los tests `..._does_not_modify_input` comparan la tabla que pasas antes y después de llamar a tu función. Encadena métodos que devuelven objetos nuevos (todos los de este capítulo lo hacen) y evita `inplace=True` y las asignaciones del tipo `prices[...] = ...` sobre el argumento.

Una advertencia del proyecto: `pyproject.toml` convierte en error cualquier `FutureWarning` o `DeprecationWarning` emitido desde `src/`. Si usas una API obsoleta de pandas, el test falla aunque el número sea correcto. Usa la API moderna: `ffill(limit=...)` y no `fillna(method="ffill")`, que en pandas 3 ya ni siquiera existe.

TODO 1.1 · simple_returns

Qué hace. Calcula el rendimiento simple diario de cada activo, para toda la tabla a la vez, sin bucles y sin rellenar huecos.

Firma.

```
def simple_returns(prices: pd.DataFrame) -> pd.DataFrame:
```

Convenciones que comprueban los tests.

- Mismo índice y mismas columnas que `prices`; la primera fila es NaN y las demás no, si no hay huecos (`test_simple_returns_first_row_nan_same_shape`).
- Valores conocidos: $A = [100, 110, 99, 99]$ da $[NaN; 0,10; -0,10; 0,0]$ (`test_simple_returns_known_values`).
- Sin precio hoy no hay rendimiento, y sin precio ayer tampoco: un hueco de un día produce dos NaN seguidos (`test_simple_returns_does_not_fill_gaps`).
- Funciona con una Series y devuelve una Series (`test_simple_returns_works_with_series`).
- No modifica la entrada (`test_simple_returns_does_not_modify_input`).

Pseudocódigo.

1. Construye "el precio de la sesión anterior": una tabla con las mismas fechas y columnas en la que cada fila contiene los precios de la fila anterior (la primera fila queda vacía).
2. Divide, celda a celda, el precio de cada fecha entre su precio anterior.
3. Resta 1 a cada celda.
4. Devuelve el resultado tal cual: no elimines la primera fila ni rellenes NaN.

Pistas.

Pista 1

La fórmula necesita dos números en la misma fila: el precio de hoy y el de la sesión anterior. El truco vectorizado es fabricar una segunda tabla "desfasada" que, alineada fecha a fecha con la original, contenga en cada fila el precio de ayer. A partir de ahí todo es aritmética celda a celda.

Pista 2

Esa tabla desfasada la da `shift`. Repasa el ejemplo de juguete: ¿qué necesitas para que la fila del martes contenga el valor del lunes? Las operaciones `/` y `-` funcionan igual en Series y en DataFrame, así que, si no usas nada específico de DataFrame (columnas, `axis=1`), el caso de la Series sale solo.

Pista 3

No uses `pct_change()`. Su comportamiento depende de la versión: en pandas 2.x rellenaba huecos por defecto con el último precio (parámetro `fill_method`, obsoleto) y emitía un `FutureWarning`, que aquí es un error; en pandas 3 ya no rellena. Constrúyelo desde la fórmula y sabrás qué pasa con cada NaN: se propaga solo y produce justo los dos NaN que pide `test_simple_returns_does_not_fill_gaps`. No necesitas `fillna` ni `dropna`.

TODO 1.2 · log_returns

Qué hace. Lo mismo que `simple_returns`, con logaritmos neperianos: $\ell_t = \ln(P_t / P_{t-1})$.

Firma.

```
def log_returns(prices: pd.DataFrame) -> pd.DataFrame:
```

Convenciones que comprueban los tests.

- Valores conocidos: $[100; 110; 121; 60,5]$ da $[NaN; \ln 1,1; \ln 1,1; \ln 0,5]$ (`test_log_returns_known_values`).

- La suma de cada columna, ignorando el NaN inicial, es $\ln(P_{\text{final}} / P_{\text{inicial}})$ (`test_log_returns_sum_is_log_of_total_growth`).
- $\exp(l) - 1$ reproduce `simple_returns` (`test_log_returns_consistent_with_simple_returns`, que llama a las dos funciones).
- Primera fila NaN y sin rellenos: [100, NaN, 110, 121] da NaN en las tres primeras filas y $\ln 1,1$ en la cuarta (`test_log_returns_first_row_nan_and_no_fill`).
- Ningún test lo exige, pero su docstring dice "mismas convenciones que `simple_returns`": conviene que también funcione con una Series.

Pseudocódigo.

1. Obtén el cociente "precio de hoy / precio de la sesión anterior" o, lo que es lo mismo, $1 +$ el rendimiento simple, que ya sabes calcular.
2. Aplica el logaritmo neperiano a cada celda, conservando fechas y columnas.
3. Devuelve el resultado sin rellenar nada.

Pistas.

Pista 1

$l_t = \ln(1 + r_t)$. Tienes dos caminos igual de válidos: partir de los precios o reutilizar `simple_returns`. Reutilizar tiene una ventaja: las convenciones (primera fila NaN, sin rellenos, Series o DataFrame) se heredan sin esfuerzo.

Pista 2

Las funciones universales de NumPy (`np.log`, `np.log1p`, `np.exp`, `np.expm1`) aplicadas a una Series o a un DataFrame trabajan celda a celda y devuelven un objeto de pandas con el mismo índice y las mismas columnas. `np.log1p(x)` calcula $\ln(1 + x)$ con más precisión que `np.log(1 + x)` cuando x es muy pequeño, que es justo el caso de los rendimientos diarios.

Pista 3

El logaritmo de NaN es NaN, así que los huecos se propagan igual que en el TODO 1.1. Vigila dos confusiones: `np.log10` es el logaritmo decimal y no sirve; y aplicar el logaritmo al rendimiento simple, en lugar de a $1 + r$, da NaN todos los días de bajada (logaritmo de un número negativo).

TODO 1.3 · clean_prices

Qué hace. Convierte la tabla en bruto de la descarga en una tabla utilizable aplicando los cinco pasos del docstring, en ese orden. `load_prices(clean=True)` la llama al cargar, así que puedes cambiar la limpieza sin volver a descargar.

Firma.

```
def clean_prices(prices: pd.DataFrame, max_ffill: int = config.MAX_FFILL_DAYS) -> pd.DataFrame:
```

Convenciones que comprueban los tests.

- Con fechas repetidas se queda la última aparición y el resultado sale ordenado: fechas [01-03, 01-01, 01-02, 01-02] con $A = [3; 1; 2; 2,5]$ dan $A = [1; 2,5; 3]$ (`test_clean_prices_drops_duplicates_keep_last_and_sorts`).
- Los precios ≤ 0 son huecos y se rellenan como cualquier otro: [10, 11, 0, 12, -1, 13] da [10, 11, 11, 12, 12, 13] (`test_clean_prices_non_positive_prices_are_gaps`).

- El relleno respeta el límite: con 7 NaN seguidos y `max_ffill=5` se rellenan los 5 primeros y los 2 últimos quedan NaN (`test_clean_prices_ffill_respects_limit`).
- Nunca rellena hacia atrás y empieza el primer día en que todos los activos tienen precio (`test_clean_prices_never_backfills_and_trims_to_common_start`).
- Una fila con todos los activos en NaN se elimina, no se rellena (`test_clean_prices_drops_rows_where_all_assets_are_nan`).
- Todas las columnas salen como `float64`, aunque entren enteros (`test_clean_prices_returns_floats`).
- No modifica la entrada, que en ese test está desordenada y tiene un duplicado, un `-1` y un NaN (`test_clean_prices_does_not_modify_input`).
- Una tabla ya limpia sale idéntica, índice incluido (`test_clean_prices_clean_data_is_unchanged`).

Pseudocódigo.

1. Marca las filas cuya fecha vuelve a aparecer más abajo y descártalas: de cada fecha sobrevive su última aparición.
2. Ordena las filas por fecha, de la más antigua a la más reciente.
3. Convierte todos los valores a coma flotante.
4. Sustituye por NaN cada celda con precio menor o igual que 0.
5. Elimina las filas en las que todas las celdas son NaN.
6. En cada columna, rellena cada NaN con el último precio válido anterior, pero solo los `max_ffill` primeros NaN de cada hueco; el resto del hueco queda NaN.
7. Para cada columna, busca su primera fecha con precio y quédate con la más tardía.
8. Devuelve las filas desde esa fecha (incluida) hasta el final.

Pistas.

Pista 1

El orden de los pasos no es estético: cambiarlo cambia el resultado. Si rellenas antes de eliminar las filas vacías, una fila sin ningún precio se convierte en un día "normal" con rendimiento 0 para todos. Si ordenas antes de quitar duplicados, "la última aparición" deja de referirse al orden de llegada, porque la ordenación por defecto no garantiza conservar el orden relativo de dos filas con la misma fecha. Si recortas el inicio antes de anular los precios ≤ 0 , un `-1` en la primera fila de un activo te da una fecha de inicio equivocada.

Pista 2

Herramientas, en el orden de los pasos. `index.duplicated(keep=...)` devuelve un array de booleanos que marca las etiquetas repetidas; con `keep="last"` marca todas las apariciones menos la última (para el índice de juguete a, b, a, c, b devuelve True, True, False, False, False), y con `~` lo niegas para seleccionar filas. `sort_index()` ordena por fecha. `astype(float)` convierte. `mask(cond)` devuelve una copia con NaN allí donde `cond` es True, al revés que `where`: sobre `[3, 8, 1]` con la condición "mayor que 5" da `[3, NaN, 1]`. `dropna(how=...)` elimina filas: con `how="any"` las que tienen algún NaN, con `how="all"` las que solo tienen NaN. `ffill(limit=...)` rellena hacia delante con un tope de NaN consecutivos.

Pista 3

Para el paso 5 del docstring necesitas la primera fecha válida de *cada columna*. Trampa: `first_valid_index()` llamado sobre el DataFrame entero devuelve la primera fila en la que *algún* activo tiene precio (la más temprana), no aquella en que los tienen todos. Llámalo columna a columna (cada columna es una Series) y quédate con la máxima. Para recortar, `.loc[fecha:]` incluye la fecha de inicio. Si una columna fuera NaN entera, `first_valid_index()` devolvería `None`: ningún test lo comprueba, pero decide si prefieres lanzar un error claro.

4. Errores típicos

Desplazar en el sentido equivocado

Usar `shift(-1)` calcula $P_{t+1}/P_t - 1$ y lo etiqueta con la fecha t : el rendimiento de mañana aparece hoy. Síntoma: el NaN sale en la última fila en vez de en la primera y todos los valores están corridos una fila. Lo detectan `test_simple_returns_known_values` y `test_simple_returns_first_row_nan_same_shape`. En un backtest, este error es look-ahead puro.

Perder la forma de la tabla

Eliminar la primera fila con `dropna()`, recortar con `[1:]` o pasar a NumPy y volver. Síntoma: una fila de menos o un índice distinto; más adelante, el motor se queja de que precios y señales no tienen las mismas fechas. Lo detecta `test_simple_returns_first_row_nan_same_shape`.

Rellenar huecos al calcular rendimientos

Usar `pct_change()` con pandas 2.x o añadir un `ffill/fillna` "por si acaso". Síntoma: aparece un rendimiento donde falta un precio, o salta un `FutureWarning` convertido en error. Lo detectan `test_simple_returns_does_not_fill_gaps` y `test_log_returns_first_row_nan_and_no_fill`. Los huecos se tratan una sola vez, en `clean_prices`, con reglas explícitas.

Código que solo sirve para DataFrame

Recorrer `prices.columns` con un bucle o usar `axis=1`. Síntoma: `AttributeError` o un resultado que no es una Series. Lo detecta `test_simple_returns_works_with_series`.

Logaritmo equivocado

Tomar el logaritmo de r en lugar de $\ln(1 + r)$, o usar `np.log10`. Síntoma: NaN todos los días de bajada en el primer caso; valores que no cuadran por un factor $\approx 2,3$ en el segundo. Lo detectan `test_log_returns_known_values` y `test_log_returns_consistent_with_simple_returns`.

Quedarse con la primera aparición

Usar `keep="first"` o `drop_duplicates()`, que compara los valores de las filas y no las fechas. Síntoma: el 2020-01-02 conserva 2,0 en vez de 2,5. Lo detecta `test_clean_prices_drops_duplicates_keep_last_and_sorts`.

Rellenar antes de eliminar las filas vacías

Síntoma: la fila sin ningún precio sobrevive con el precio del día anterior y la tabla tiene 3 filas en vez de 2. Lo detecta `test_clean_prices_drops_rows_where_all_assets_are_nan`.

Eliminar filas con algún NaN

Usar `dropna()` a secas, que por defecto es `how="any"`. Síntoma: desaparecen las fechas con un único precio ausente o con un precio ≤ 0 , que deberían rellenarse. Lo detectan `test_clean_prices_non_positive_prices_are_gaps` y `test_clean_prices_ffill_respects_limit`.

Rellenar sin límite o hacia atrás

`ffill()` sin `limit`, `bfill()` o `fillna` con un valor fijo. Síntoma: huecos largos rellenos por completo, o precios en fechas en que el activo aún no cotizaba. Lo detectan `test_clean_prices_ffill_respects_limit` y `test_clean_prices_never_backfills_and_trims_to_common_start`.

Recortar en la fecha más temprana

Usar `first_valid_index()` sobre el DataFrame entero, o el mínimo de las fechas por columna. Síntoma: la tabla empieza con NaN en algún activo. Lo detecta `test_clean_prices_never_backfills_and_trims_to_common_start`.

Modificar la entrada u olvidar el tipo

Asignar sobre `prices` o usar `inplace=True` cambia la tabla de quien te llama (`test_clean_prices_does_not_modify_input`). Olvidar `astype(float)` deja columnas enteras cuando no hay nada que convertir en NaN (`test_clean_prices_returns_floats`).

5. Preguntas y ejercicios

1. Una acción cierra a 300 USD y al día siguiente hace un split 3:1 y cierra a 101 USD. ¿Qué rendimiento ve un backtest con precios sin ajustar ese día? ¿Y con precios ajustados?
2. Una acción cierra a 80 USD, paga al día siguiente un dividendo de 1,60 USD (ex-dividendo) y cierra a 79 USD. Calcula el rendimiento de la cotización, el factor de ajuste y el rendimiento ajustado.
3. Cartera 60/40 en dos activos que un día rinden +3 % y -2 %. Calcula el rendimiento de la cartera con simples y la media ponderada de los logarítmicos. ¿Cuál describe lo que le pasa a tu dinero?
4. Un activo sube un 20 % y luego baja un 20 %. ¿Cuánto vale la media de los rendimientos simples? ¿Y el capital final? Relaciona la diferencia con la fórmula del volatility drag.
5. ¿Por qué un relleno hacia delante de 5 sesiones es razonable y uno de 60 no? Piensa en qué le pasa a la volatilidad medida y a una media móvil calculada durante el hueco.
6. Describe un caso concreto en el que un `bfill` de los precios haga ganar a una estrategia dinero que no habría ganado en la realidad.
7. ¿Por qué `simple_returns` no rellena los NaN si `clean_prices` ya ha decidido qué rellenar? ¿Qué pasaría si ambas funciones rellenaran?
8. El sesgo de supervivencia, ¿infla o deprime los resultados de un backtest? ¿Afecta igual a SPY que a AAPL? ¿Qué datos necesitarías para eliminarlo?
9. Ejercicio (notebook `01_exploracion.ipynb`). Construye a mano una tabla sucia con tres activos que tenga una fecha repetida, un precio 0, una fila entera de NaN, un hueco de 7 sesiones y un activo que empiece tarde. Predice en papel la salida de `clean_prices` con `max_ffill=5` y compárala con la real.

10. Ejercicio (notebook). Sobre los datos descargados, calcula $r - \ell$ para todos los días y activos. ¿Cuál es la diferencia media en bps? ¿En qué día y en qué activo es máxima? Comprueba que se parece a $r^2/2$.

6. Checkpoint

```
python -m pytest tests/test_data.py
python -m pytest -k simple_returns
python -m pytest -k log_returns
python -m pytest -k clean_prices
python -m pytest tests/test_data.py -rs
python -m pytest tests/test_data.py --todo-fail
```

Ejecuta los comandos desde la raíz del repo con el entorno virtual activado. El primero es el checkpoint del capítulo; los tres con `-k` seleccionan solo los tests cuyo nombre contiene ese texto, útil para ir TODO a TODO. Ojo: `-k simple_returns` también selecciona `test_log_returns_consistent_with_simple_returns`, que llama a las dos funciones y seguirá saliendo como skipped mientras el TODO 1.2 esté pendiente.

El autocorrector (`tests/conftest.py`) trata un `NotImplementedError` como **skipped**, no como fallo. Antes de empezar verás `17 skipped` y, al final de la salida, una sección "TODO pendientes" con cada TODO y cuántos tests lo esperan. Un TODO que hayas implementado mal no sale como skipped: sale como **FAILED**, con el mensaje del `assert` que lo explica. La opción `-rs` muestra el motivo de cada skip y `--todo-fail` cuenta los TODO pendientes como fallos, por si quieres ver el rojo completo.

Resultado esperado al terminar el capítulo: `17 passed` en `tests/test_data.py`, sin skipped. Después, `python -m src.data` (si no lo has hecho) y el notebook `01_exploracion.ipynb` con `load_prices(clean=True)` ya funcionan.